


SIMATS
ENGINEERING

SIMATS
Sreevith Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 Cloud Computing and Big Data Analytics

CO1 AT2 - Digital Concept Map and Service Model Matrix

Individual Student Submission Template

Student and Faculty Details

Student Name	DENVAR JOE H	Register Number	192511438
Class / Section		Slot	C
Course Code	CSA1571	Course Title	Cloud Computing and Big Data Analytics
Capstone Title	AquaGuard Cloud Water Quality Reporter With Sensor Analytics and Public Alert		
Submission Date	16/07/2026		
Faculty Name	Dr.Rajesh Babu.C	Employee ID	
Department	BE.CSE		

Assessment Brief

Course Outcome	CO1 - Explain the evolution, architecture, characteristics, service models, and emerging paradigms of cloud computing.
Assessment Type	Individual concept mapping, service-model responsibility analysis, and capstone-cloud interpretation.
Assessment Focus	Each map must show key nodes, link labels, relationship meaning, and a logical hierarchy. The response must connect cloud fundamentals to the student's own perception of the capstone title.
Submission Mode	Editable DOCX + exported diagram evidence + GitHub repository link submitted in Google Classroom.
Permitted Tools	draw.io / diagrams.net, Lucidchart, Miro, Canva, PowerPoint, Google Drawings, or equivalent diagramming tool.
Individual Rule	Every student must submit their own concept map, matrix, capstone-cloud mapping, and reflection. The capstone title may be shared by a team, but the assessment answer must not be common.
Total Marks	100 marks

Code of Conduct and Student Assurance

Academic Integrity Statement

This assessment must be completed individually. Students may discuss the capstone domain with their team, but the concept map, service model matrix, explanations, GitHub evidence, and reflection submitted here must be individually prepared.

Student Assurance	I confirm that this submission represents my own understanding of CO1 concepts and my individual interpretation of the selected capstone title.
Tool Use	I have used digital tools only for diagram preparation, organization, and presentation. I have not copied another student's diagram, matrix, or explanation.
Evidence	I have uploaded the required files to my GitHub repository and submitted the repository link / evidence link through Google Classroom.

Student Signature	
Date	

Procedure for Preparing the Digital Concept Map

Use the following procedure to prepare the diagram and upload evidence. Students who already know another diagramming tool may use it, but the exported evidence and source file must still be submitted.

1. Open <https://app.diagrams.net/> or the desktop version of draw.io / diagrams.net.
2. Choose Device, Google Drive, or OneDrive as the save location according to availability.
3. Create a blank diagram and name the file as CSA15_CO1_AT2_RegisterNumber_Concept_Map.drawio.
4. Place the capstone title at the center or as a clearly visible anchor node.
5. Add major cloud nodes: cloud definition, cloud evolution, cloud architecture, service models, deployment models, virtualization, web services, SOAP, REST, APIs, elasticity, scalability, resource pooling, measured service, and emerging paradigms.
6. Connect nodes with labeled arrows such as uses, depends on, enables, scales through, communicates through, belongs to, supports, provides, consumes, or controls.
7. Use colors meaningfully: one color for service models, one for cloud characteristics, one for API/web-service concepts, and one for capstone-specific elements.
8. Export the final diagram as PNG or PDF. Keep the editable .drawio source file also.
9. Paste the exported diagram image or screenshot in this DOCX under Task 1.
10. Upload the DOCX, exported PNG/PDF, and .drawio file to GitHub, then submit the GitHub link in Google Classroom.

GitHub Submission Procedure

Step	Required Action	Expected Evidence
1	Create or use your individual GitHub account.	GitHub profile link.
2	Create a repository named CSA15-CO-Assessments-RegisterNumber or use the repository instructed by faculty.	Repository URL.
3	Create a folder named CO1_AT2 inside the repository.	Folder visible in repository.
4	Upload this completed DOCX, exported diagram PNG/PDF, and editable .drawio source file.	At least three evidence files uploaded.
5	Add a short README.md explaining the capstone title and files submitted.	README visible in the AT2 folder.
6	Submit the GitHub folder link in Google Classroom.	GCR submission with repository/folder link.

Assessment Questions and Student Response Sections

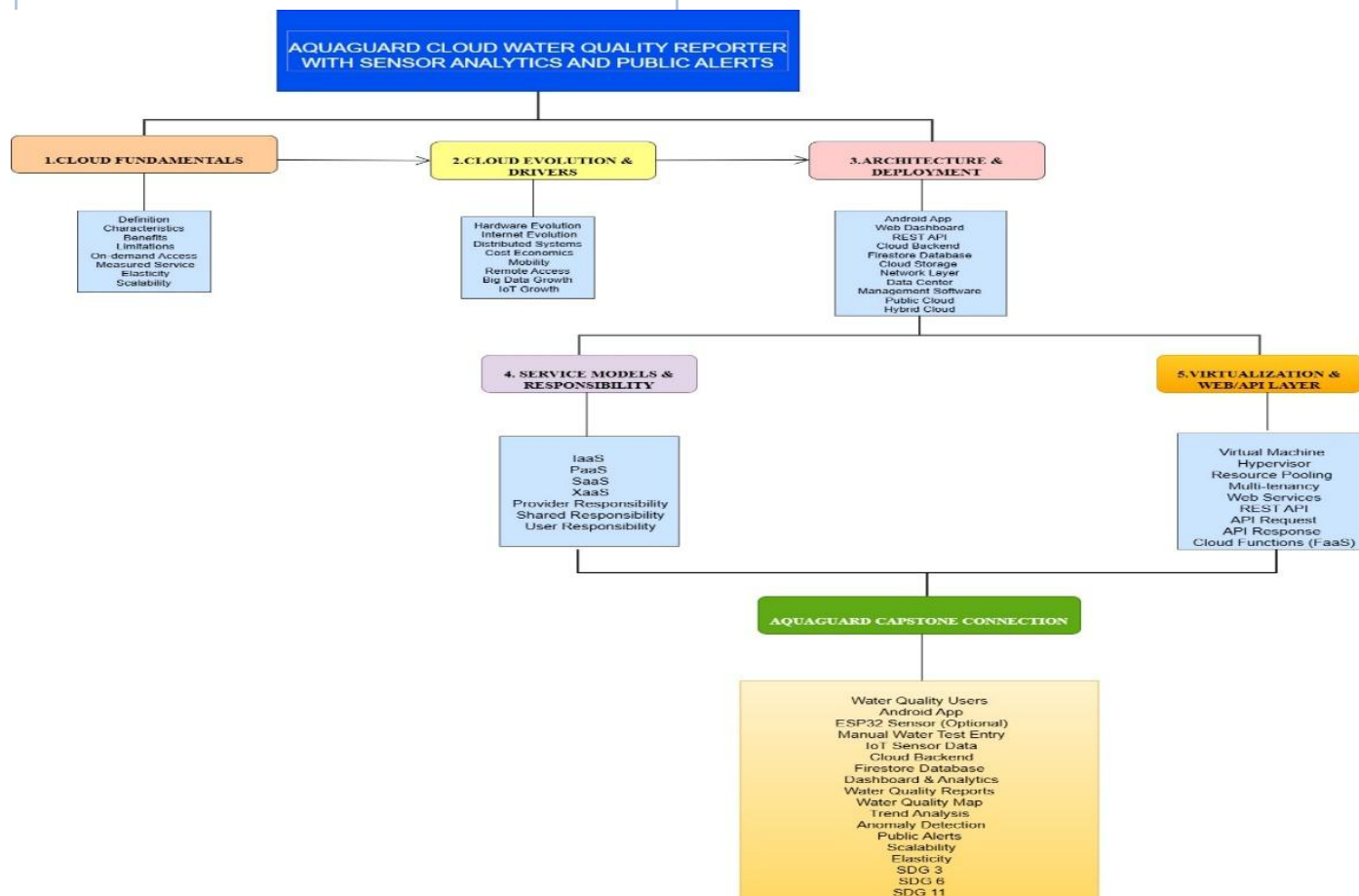
Important

The capstone project may be developed by a team, but every response below must be written from the individual student's own perspective. Do not submit a common team answer.

Task 1 - Digital Cloud Concept Map

Create one integrated digital concept map for CO1. The map must show key nodes, link labels, relationship meaning, and a logical hierarchy. It must also connect the concepts to your selected capstone title from your individual perspective.

Mandatory Concept Group	Minimum Concepts / Nodes to Include
Cloud Fundamentals	Definition, characteristics, benefits, limitations, on-demand access, measured service.
Cloud Evolution and Drivers	Hardware evolution, internet software evolution, distributed systems, cost economics, mobility, remote access, big data growth.
Architecture and Deployment	Front-end, back-end, network layer, data center, management software, public/private/hybrid/community cloud.
Service Models and Responsibility	IaaS, PaaS, SaaS, XaaS, provider responsibility, user responsibility, shared responsibility.
Virtualization and Web/API Layer	Virtual machine, hypervisor, abstraction, resource pooling, multi-tenancy, web services, SOAP, REST, API request/response.
Capstone Connection	Your capstone users, app/module, cloud backend, database/storage, dashboard/analytics, scalability and elasticity idea.
Paste exported concept map image / screenshot here. Resize neatly within this white space.	



Brief explanation of your concept map: explain the most important nodes, link labels, hierarchy, and capstone connection.

This concept map presents the AquaGuard Cloud Water Quality Reporter and its cloud computing architecture. It begins with cloud fundamentals, followed by cloud evolution, architecture and deployment, service models, and virtualization with REST APIs. These concepts are connected to the AquaGuard system, where an Android app, optional ESP32 sensors, cloud backend, Firestore database, dashboard, and analytics work together to provide real-time water quality monitoring, trend analysis, and public alerts. The system is secure, scalable, and supports SDG 3, SDG 6, and SDG 11.

Task 2 - Service Model Matrix

Complete the matrices by making technical decisions for your capstone project. Select only the rows that are relevant to your capstone design. If a row is not used, write a short technical reason instead of leaving it blank.

Your answer should show how responsibility shifts between cloud provider and student development team, and how the chosen services support the capstone workflow, API design, storage, security, scalability, and evidence submission.

Task 2A - Service Model Responsibility Matrix

Service Model	Capstone Role / Module	Provider Responsibility	Student Responsibility	Evidence to Submit
IaaS	Cloud Infrastructure	Provides virtual machines, storage, networking	Configure cloud resources and deploy application	Cloud architecture screenshot
PaaS	Backend & API Development	Provides runtime, database, deployment platform	Develop REST APIs and backend logic	API code and backend screenshots
SaaS	Android App & Web Dashboard	Hosts application and dashboard	Design user interface and reports	Dashboard screenshots
XaaS / Specialized Service	Notification & Map Services	Provides Email, SMS, Maps, Analytics APIs	Integrate cloud services	Notification demo

Task 2B - Capstone Cloud Service Decision Matrix

Cloud Layer / Function	Selected Tool or Service	Data / Resource Handled	API or Integration Path	Security / Scaling Note
Client access and UI	Android App	Water quality readings, user requests	REST API	User authentication and HTTPS
API / backend logic	Python (Flask/FastAPI)	Data validation and processing	REST API	Auto-scaling cloud backend
Authentication and identity	Firebase Authentication	User login and access control	Firebase Auth API	Secure user authentication
Database / structured records	Firestore Database	Sensor data, manual test records, user details	Firestore API	Encrypted cloud database
File / object storage	Firebase Storage	Water quality reports and documents	Storage API	Secure cloud storage
Monitoring / logs / usage metrics	Firebase Analytics	Application logs and usage statistics	Analytics API	Real-time monitoring
Analytics / reporting output	Python Analytics Engine	Trend analysis, anomaly detection, public reports	Analytics API	Scalable analytics processing
Technical justification: Explain why the selected cloud service pattern fits your capstone project's users, data, API flow, scalability, and security needs. The AquaGuard Cloud Water Quality Reporter uses cloud services to provide secure, scalable, and real-time water quality monitoring. The Android application sends sensor readings and manual test results to the cloud backend through REST APIs. Firebase Authentication ensures secure user access, while				

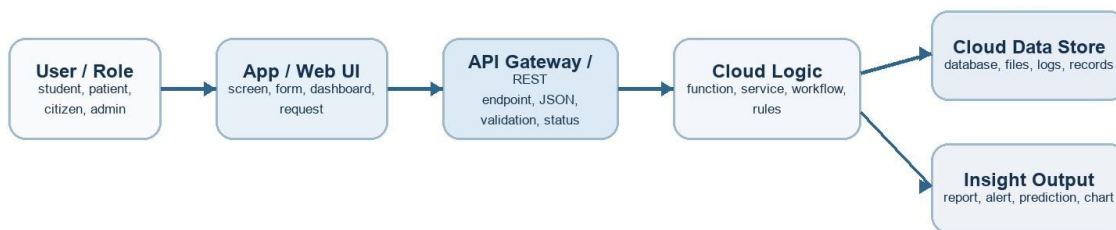
Firestore stores structured water quality data. Firebase Storage maintains reports and supporting files, and the analytics engine generates trend analysis, anomaly detection, and public alerts. Cloud services provide high availability, automatic scalability, secure data management, and reliable performance for multiple users and locations.

Task 3 - Capstone Cloud Flow Mapping

Study the sample architecture-flow template below. Redraw it in draw.io / diagrams.net using your own capstone architecture. Replace every generic label with your actual user role, screen, API endpoint, backend logic, cloud storage, analytics output, security control, and scalability or elasticity decision.

Sample Capstone Cloud Flow Template

Replicate this structure using your own capstone screens, APIs, cloud services, data stores, and outputs.



Cross-cutting cloud decisions to label in your own diagram

Authentication / IAM

Who is allowed to access each API and data item?

Scalability

Which component must handle growing users or records?

Elasticity

Which component can expand or shrink based on demand?

Monitoring

What logs, usage metrics, or errors will be observed?

Security

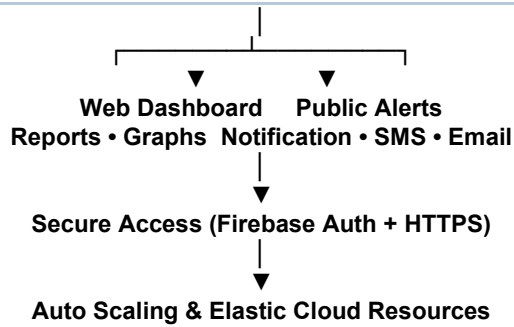
Where are validation, privacy, backup, and access control applied?

Student instruction: redraw this flow in draw.io/diagrams.net and replace every label with your own capstone architecture.

Student Drawing Requirement

Do not copy the sample as it is. Use it as a layout model, then create your own capstone-specific flow diagram with meaningful labels, arrows, data movement, API names, and cloud service choices.

Flow Component	Student Entry
User role and action	Water Quality Officer / Public User enters water quality data through the Android App or ESP32 sensor.
Frontend / app screen	Android Application and Web Dashboard for data entry, monitoring, reports, and alerts.
API endpoint and method	POST /api/water-quality (Submit water quality data using REST API).
Backend processing rule	Validate sensor/manual data, process readings, store data, perform trend analysis and anomaly detection.
Cloud database / storage operation	Firestore Database stores water quality records, user details, historical data, and reports. Firebase Storage stores report files and documents.
Response, alert, report, or dashboard output	Dashboard displays real-time water quality status, trend graphs, maps, monthly reports, and sends public alerts if unsafe water is detected.
Security, scalability, and elasticity labels added in diagram	Firebase Authentication, HTTPS Encryption, Role-Based Access Control, Auto Scaling, Elastic Cloud Resources, Secure Cloud Storage.
Paste your redrawn capstone cloud-flow diagram here. Water Quality Officer / Public User ↓ Android App / ESP32 Sensor ↓ POST /api/water-quality (REST API) ↓ Cloud Backend (Python / Firebase) ↓ Validate & Process Water Quality Data ↓ Firestore Database / Firebase Storage ↓ Analytics Engine (Trend Analysis & Anomaly Detection)	



Brief explanation: explain one complete request-response path from user action to cloud response.

The AquaGuard Cloud Water Quality Reporter begins when a user or water quality officer submits water quality data through the Android application or when an optional ESP32 sensor automatically sends readings. The data is transferred to the cloud backend using a REST API, where it is validated and securely stored in the Firestore Database. The analytics engine processes the stored data to perform trend analysis and anomaly detection. The processed information is displayed on the Web Dashboard as real-time reports, graphs, and water quality maps. If unsafe water quality is detected, the system automatically sends public alerts through notifications. Cloud security features such as Firebase Authentication and HTTPS encryption protect user data, while auto-scaling and elastic cloud resources ensure reliable performance as the number of users and sensor data increases.

Task 4 - Individual Reflection

Answer the following from your own perspective. Avoid team-common wording.

1. What is the strongest cloud concept represented in your capstone project?

The strongest cloud concept represented in my capstone project is real-time cloud computing with centralized data storage and analytics. The AquaGuard system collects water quality data from optional ESP32 sensors and manual test reports through the Android application. The data is securely stored in the cloud database, where it is processed for trend analysis and anomaly detection. The cloud enables users to access reports and dashboards from anywhere while ensuring high availability, reliability, and efficient resource management. This makes the system suitable for continuous monitoring and timely public alerts.

2. Where will elasticity or scalability become important in your capstone product?

Elasticity and scalability become important when the number of users, monitoring locations, and sensor devices increases. As more water quality data is collected from different areas, the cloud can automatically allocate additional computing and storage resources without affecting system performance. This ensures that the AquaGuard platform continues to provide fast data processing, real-time dashboards, and public notifications even during periods of heavy usage. Cloud scalability also allows the system to expand easily as new locations are added.

3. Which API or web-service idea is most relevant to your project and why?

The REST API is the most relevant web-service technology used in my project. It provides secure communication between the Android application, web dashboard, cloud backend, Firestore database, and analytics engine. REST APIs allow sensor readings and manual water quality reports to be transmitted in real time for processing and storage. They also enable dashboards, reports, and public alerts to be updated instantly. Because of its simplicity, flexibility, and wide compatibility, REST API is the best choice for integrating all components of the AquaGuard system.

4. What did you personally understand better after preparing this concept map and matrix?

After preparing the concept map and service model matrix, I developed a better understanding of cloud computing concepts such as cloud architecture, deployment models, IaaS, PaaS, SaaS, virtualization, REST APIs, scalability, elasticity, and cloud security. I also learned how these technologies work together to build a real-time water quality monitoring system. This assessment improved my knowledge of designing cloud-based applications and helped me understand how cloud services can be used to solve real-world environmental and public health problems.

Evaluation Rubric - 100 Marks

Criteria	Marks	Expected Standard
Concept identification	20	Major CO1 concepts are accurately identified: cloud fundamentals, characteristics, evolution, service models, deployment models, virtualization, web services, SOAP/REST, APIs, elasticity, and scalability.
Relationship mapping	20	Links between concepts are accurate, meaningful, and labeled. The map should explain how concepts depend on, enable, consume, provide, scale, or control one another.
Hierarchy and organization	15	Concepts are arranged in a logical hierarchy with clear grouping, readable flow, and no disconnected keyword clusters.

Technical relevance	15	The map strongly reflects cloud course content and avoids decorative or generic non-technical content.
Service model matrix	15	IaaS, PaaS, SaaS, and XaaS are compared with provider responsibility, user responsibility, examples, capstone usage, and risks or limitations.
Capstone alignment and individual reflection	10	The student's own capstone interpretation is visible in the concept map, flow mapping, justification, and reflection. Team-common answers are avoided.
Visual clarity and digital evidence	5	Diagram is neat, readable, exported properly, and supported with GitHub evidence including source and exported files.

Faculty Evaluation Record

Evaluator Name		Marks Awarded	
Observation		Signature	